

RAPPORT D'ARCHITECTURE LOGICIELLE

Système de Gestion de Tâches — *Task Manager*

Du Monolithe Modulaire aux Microservices

UE : Architecture Logicielle — Licence 3

Étudiant : **Matar Gueye**

Technologie : Laravel (PHP) · PostgreSQL

Année académique : 2024–2025

Table des Matières

- 1. Introduction et Présentation du Projet
- 2. Analyse Métier
 - 2.1 Acteurs du Système
 - 2.2 Fonctionnalités Principales
 - 2.3 Exigences Fonctionnelles
 - 2.4 Exigences Non Fonctionnelles
- 3. Architecture Decision Records (ADR)
 - ADR-001 : Choix de l'Architecture Monolithique Modulaire
 - ADR-002 : Choix de Laravel (PHP)
 - ADR-003 : Choix de PostgreSQL
 - ADR-004 : Communication Asynchrone via RabbitMQ
 - ADR-005 : Migration vers les Microservices
- 4. Phase 1 — Conception Architecturale
 - 4.1 Vue Globale du Système (Contexte C4)
 - 4.2 Découpage en Modules
 - 4.3 Schéma d'Architecture
- 5. Phase 2 — Monolithe Modulaire
 - 5.1 Structure du Projet Laravel
 - 5.2 Architecture en Couches
 - 5.3 Patterns Utilisés
 - 5.4 Modèle de Données
 - 5.5 API REST & Documentation Swagger
 - 5.6 Sécurité — Authentification JWT
- 6. Phase 3 — Transition vers les Microservices
 - 6.1 Identification des Bounded Contexts
 - 6.2 Découpage des Services
 - 6.3 API Gateway
 - 6.4 Communication Synchrone & Asynchrone
 - 6.5 Séparation des Bases de Données

6.6 Résilience et Observabilité

7. Sécurité

8. Gestion de la Dette Technique

9. Difficultés Rencontrées

10. Conclusion

1. Introduction et Présentation du Projet

Ce rapport présente la conception et l'évolution architecturale d'un Système de Gestion de Tâches (Task Manager), développé dans le cadre du module d'Architecture Logicielle en Licence 3. Le projet a pour objectif de démontrer une maîtrise complète des architectures logicielles modernes, depuis l'analyse métier jusqu'à la mise en place d'une architecture microservices distribuée.

Le Task Manager est une application web permettant à des utilisateurs de créer, organiser, assigner et suivre des tâches. Le système intègre une gestion des catégories, des priorités, des délais et un mécanisme de notifications.

La démarche adoptée suit une progression en trois phases : d'abord une analyse architecturale approfondie, ensuite la construction d'un monolithe modulaire propre avec Laravel, et enfin la transition vers une architecture microservices avec une API Gateway, des communications synchrones et asynchrones.

2. Analyse Métier

2.1 Acteurs du Système

Le système identifie trois types d'acteurs principaux interagissant avec la plateforme :

- **Utilisateur Standard** : crée et gère ses propres tâches, les organise par catégories, définit des priorités et reçoit des notifications.
- **Administrateur** : gère les utilisateurs, supervise l'ensemble des tâches, accède aux métriques et configure les paramètres globaux du système.
- **Système (acteur technique)** : déclenche les notifications automatiques, envoie les rappels d'échéance et effectue les opérations planifiées.

2.2 Fonctionnalités Principales

Module	Fonctionnalité	Priorité
Users	Inscription / Connexion / Profil	Haute
Users	Authentification JWT	Haute
Tasks	Créer / Modifier / Supprimer une tâche	Haute
Tasks	Assigner une priorité et une date limite	Haute
Tasks	Changer le statut (Todo/In Progress/Done)	Haute
Categories	Créer / Gérer les catégories	Moyenne
Categories	Associer une catégorie à une tâche	Moyenne
Notifications	Notification à la création d'une tâche	Moyenne
Notifications	Rappel avant échéance	Moyenne

2.3 Exigences Fonctionnelles

- EF-01 : L'utilisateur doit pouvoir s'inscrire avec un email unique et un mot de passe.
- EF-02 : L'utilisateur doit pouvoir se connecter et recevoir un token JWT.
- EF-03 : L'utilisateur doit pouvoir créer, modifier, archiver et supprimer des tâches.
- EF-04 : Chaque tâche doit avoir un titre, une description, une priorité, un statut et une date limite.
- EF-05 : L'utilisateur doit pouvoir filtrer et rechercher ses tâches par statut, priorité ou catégorie.
- EF-06 : Le système doit envoyer une notification lors de la création d'une tâche.
- EF-07 : Le système doit envoyer un rappel 24h avant l'échéance d'une tâche.
- EF-08 : L'administrateur doit accéder à un tableau de bord avec des statistiques.

2.4 Exigences Non Fonctionnelles

Critère	Exigence	Mesure
Performance	Temps de réponse API	< 200 ms (95e percentile)
Disponibilité	Uptime du système	99.5% (hors maintenance)
Sécurité	Authentification	JWT + HTTPS obligatoire
Scalabilité	Charge simultanée	500 utilisateurs actifs
Maintenabilité	Couverture tests unitaires	> 70%
Observabilité	Logs structurés	Format JSON centralisé

3. Architecture Decision Records (ADR)

Les ADR (Architecture Decision Records) documentent les décisions architecturales importantes prises au cours du projet. Chaque ADR présente le contexte, la décision, les alternatives envisagées et les conséquences attendues.

ADR-001 : Choix d'une Architecture Monolithique Modulaire

Champ	Détail
Statut	Accepté
Date	2025
Décideur	Matar Gueye

Contexte

Le projet est un système de gestion de tâches de taille moyenne. Pour un premier déploiement, la complexité d'une architecture microservices complète serait prématurée.

Décision

Adopter un monolithe modulaire avec Laravel, organisé en modules métier faiblement couplés. Cette approche permet de commencer simplement tout en préparant une future migration vers les microservices.

Alternatives envisagées

- Microservices dès le départ : rejeté car complexité opérationnelle trop élevée.
- Monolithe classique non modulaire : rejeté car difficile à faire évoluer.

Conséquences

- + Développement rapide et déploiement simplifié.
- + Code organisé et maintenable grâce aux modules.
- - Un seul point de déploiement, scalabilité limitée.

ADR-002 : Choix de Laravel (PHP)

Champ	Détail
Statut	Accepté
Date	2025

Contexte

Choix du framework backend pour l'implémentation du monolithe modulaire.

Décision

Utiliser Laravel (PHP) grâce à sa maturité, son écosystème riche (Sanctum pour JWT, Eloquent ORM, Queues, Events), et sa structure MVC facilitant l'organisation modulaire.

Alternatives envisagées

- NestJS : bon choix mais moins maîtrisé par le développeur.
- Spring Boot : excellent mais verbosité Java plus contraignante.

Conséquences

- + Développement rapide grâce aux fonctionnalités intégrées de Laravel.
- + Eloquent ORM simplifie les interactions avec la base de données.
- - Performance moindre comparée à Go ou Java pour des charges très élevées.

ADR-003 : Choix de PostgreSQL

Contexte

Sélection du système de gestion de base de données pour l'application.

Décision

Utiliser PostgreSQL comme base de données principale grâce à sa robustesse, son support des transactions ACID, ses performances et sa compatibilité avec Laravel.

Conséquences

- + Transactions ACID garantissant la cohérence des données.
- + Support natif du JSON pour des champs flexibles.
- - Configuration et administration plus complexes que MySQL.

ADR-004 : Communication Asynchrone via RabbitMQ

Contexte

Besoin de découpler les services de notifications des services métier principaux.

Décision

Utiliser RabbitMQ comme message broker pour les communications asynchrones entre le service de tâches et le service de notifications.

Conséquences

- + Découplage fort entre les services.
- + Résilience : les messages sont conservés si le service est temporairement indisponible.
- - Complexité opérationnelle supplémentaire (gestion du broker).

ADR-005 : Stratégie de Migration vers les Microservices

Contexte

Après validation du monolithe, besoin d'évoluer vers une architecture plus scalable.

Décision

Adopter le pattern Strangler Fig : extraire progressivement les modules du monolithe en services indépendants, en commençant par le service de notifications (le moins couplé), puis le service utilisateurs, et enfin le service de tâches.

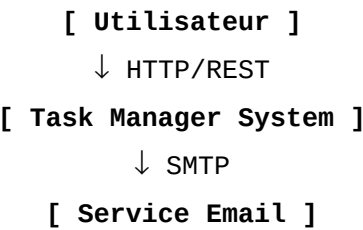
Conséquences

- + Migration progressive sans interruption de service.
- + Possibilité de valider chaque service extrait avant de continuer.
- - Période de cohabitation monolithe/microservices à gérer.

4. Phase 1 — Conception Architecturale

4.1 Vue Globale du Système (Contexte C4 — Niveau 1)

Le diagramme de contexte C4 présente le système Task Manager et ses interactions avec les acteurs externes :



Le système Task Manager est l'acteur central. Il reçoit les requêtes des utilisateurs via une interface REST et interagit avec un service email externe pour l'envoi des notifications.

4.2 Découpage en Modules Métier

Le système est organisé en quatre modules métier cohérents et faiblement couplés :

Module	Responsabilité	Dépendances
Users	Gestion des comptes, authentification	Aucune
Tasks	CRUD des tâches, gestion des statuts	Users, Categories
Categories	Organisation et classement des tâches	Users
Notifications	Envoi d'alertes et rappels	Tasks, Users

4.3 Schéma d'Architecture Monolithique Modulaire

■■■■■■■■■■ ■■■■■■

■■■■■■■■■■ ■■■■■■■■■■

5. Phase 2 — Monolithe Modulaire

5.1 Structure du Projet Laravel

Le projet Laravel est organisé selon une architecture en couches respectant les principes SOLID et les patterns Repository et Service Layer :

```
app/
■■■■ Http/
■ ■■■■ Controllers/ # Couche Présentation
■ ■ ■■■■ AuthController.php
■ ■ ■■■■ TaskController.php
■ ■ ■■■■ CategoryController.php
■ ■ ■■■■ NotificationController.php
■ ■■■■ Requests/ # Validation (Form Requests)
■ ■■■■ Resources/ # DTO — API Resources
■■■■ Services/ # Couche Métier
■ ■■■■ TaskService.php
■ ■■■■ UserService.php
■ ■■■■ NotificationService.php
■■■■ Repositories/ # Couche Accès Données
■ ■■■■ Interfaces/
■ ■ ■■■■ TaskRepositoryInterface.php
■ ■■■■ TaskRepository.php
■■■■ Models/ # Entités Eloquent
■ ■■■■ User.php
■ ■■■■ Task.php
■ ■■■■ Category.php
■■■■ Events/ + Listeners/ # Communication asynchrone interne
```

5.2 Architecture en Couches

Le monolithe respecte une architecture en 4 couches strictement séparées :

Couche	Rôle	Composants Laravel
Présentation	Exposer les endpoints REST	Controllers, FormRequests, Resources
Application	Orchestrer la logique métier	Services
Domaine	Règles métier et entités	Models, Events
Infrastructure	Accès aux données et services externes	Repositories, Queues, Mailers

5.3 Patterns de Conception Utilisés

Pattern	Où	Bénéfice
Repository	TaskRepository, UserRepository	Découplage BDD / Logique
Service Layer	TaskService, NotificationService	Logique métier centralisée

DTO (API Resource)	TaskResource, UserResource	Contrôle de l'exposition API
Dependency Injection	ServiceProvider, Constructeurs	Inversion de dépendance
Observer/Event	TaskCreated Event + Listener	Découplage notifications
Strategy	Priorité des tâches	Comportement interchangeable

5.4 Modèle de Données

Le schéma de base de données comprend les tables suivantes :

Table	Colonnes Principales	Relations
users	id, name, email, password, role	hasMany tasks
tasks	id, title, desc, status, priority, due_date, user_id, category_id	belongsTo user, category
categories	id, name, color, user_id	hasMany tasks
notifications	id, user_id, task_id, type, message, read_at	belongsTo user, task

5.5 API REST & Documentation Swagger

L'API REST expose les endpoints suivants, documentés via Laravel OpenAPI/Swagger :

Méthode	Endpoint	Description	Auth
POST	/api/auth/register	Inscription utilisateur	Non
POST	/api/auth/login	Connexion + token JWT	Non
GET	/api/tasks	Liste des tâches (paginée)	Oui
POST	/api/tasks	Créer une tâche	Oui
GET	/api/tasks/{id}	Détail d'une tâche	Oui
PUT	/api/tasks/{id}	Modifier une tâche	Oui
DELETE	/api/tasks/{id}	Supprimer une tâche	Oui
GET	/api/categories	Liste des catégories	Oui
POST	/api/categories	Créer une catégorie	Oui
GET	/api/notifications	Notifications utilisateur	Oui

5.6 Sécurité — Authentification JWT

L'authentification est gérée via Laravel Sanctum (tokens API). Le flux est le suivant :

1. L'utilisateur envoie ses credentials via POST /api/auth/login.
2. Le système valide les credentials et génère un token Bearer JWT.

- 3. Le token est inclus dans le header Authorization de chaque requête suivante.
- 4. Le middleware auth:sanctum valide le token à chaque requête protégée.
- 5. Les tokens expirent après 24h pour des raisons de sécurité.

```
// Exemple de protection dans les routes
Route::middleware('auth:sanctum')->group(function () {
Route::apiResource('tasks', TaskController::class);
Route::apiResource('categories', CategoryController::class);
});
```

6. Phase 3 — Transition vers les Microservices

6.1 Identification des Bounded Contexts

L'analyse du domaine métier identifie trois bounded contexts distincts, correspondant chacun à un futur microservice :

Bounded Context	Responsabilité	Entités
Identity Context	Gestion des identités et accès	User, Token, Role
Task Context	Gestion des tâches et catégories	Task, Category, Status
Notification Context	Alertes et communications	Notification, Template

6.2 Découpage en Microservices

Le système est découpé en trois microservices distincts, chacun déployable indépendamment :

Service	Port	Tech	Base de Données
user-service	8001	Laravel	PostgreSQL (db_users)
task-service	8002	Laravel	PostgreSQL (db_tasks)
notification-service	8003	Laravel	PostgreSQL (db_notif)

6.3 API Gateway

Une API Gateway (implémentée avec Laravel ou Nginx) fournit un point d'entrée unique pour tous les clients. Elle assure les fonctions suivantes :

- Routage : redirige les requêtes vers le bon microservice selon l'URL.
- Authentification centralisée : valide le token JWT avant de transmettre la requête.
- Rate Limiting : limite le nombre de requêtes par utilisateur/IP.
- Load Balancing : distribue la charge entre les instances des services.

```
// Règles de routage API Gateway
/api/users/* → user-service:8001
/api/tasks/* → task-service:8002
/api/notifications/* → notification-service:8003
```

6.4 Communication Synchrone & Asynchrone

Communication Synchrone (HTTP REST)

Utilisée pour les requêtes nécessitant une réponse immédiate :

- Le task-service interroge le user-service pour vérifier qu'un utilisateur existe.
- L'API Gateway interroge les services pour assembler les réponses.

```
// task-service appelle user-service via HTTP
$response = Http::get('http://user-service:8001/api/users/' . $userId);
if ($response->failed()) throw new UserNotFoundException();
```

Communication Asynchrone (RabbitMQ)

Utilisée pour les opérations pouvant être traitées en différé :

- task-service publie un événement TaskCreated dans RabbitMQ.
- notification-service consomme l'événement et envoie la notification.

```
// task-service publie un message
RabbitMQ::publish('task.created', [
    'task_id' => $task->id,
    'user_id' => $task->user_id,
    'title' => $task->title,
]);

// notification-service consomme le message
RabbitMQ::consume('task.created', function($message) {
    NotificationService::send($message['user_id'], $message);
});
```

6.5 Séparation des Bases de Données

Chaque microservice possède sa propre base de données (Database per Service pattern). Cette séparation garantit l'autonomie et évite le couplage au niveau des données.

Problématique	Solution adoptée
Jointures cross-service	Appels HTTP synchrones entre services
Transactions distribuées	Pattern SAGA — compensation en cas d'échec
Cohérence des données	Cohérence éventuelle via messages RabbitMQ
Duplication de données	Copie locale des données essentielles

6.6 Résilience et Observabilité

Mécanismes de Résilience

Mécanisme	Description	Implémentation
Timeout	Limite le temps d'attente d'une réponse	Http::timeout(5)
Retry	Retente automatiquement en cas d'échec	Http::retry(3, 100)
Circuit Breaker	Coupe le circuit après N échecs consécutifs	Package spatie/laravel-circuit-breaker
Health Check	Endpoint de vérification de santé	GET /api/health

Observabilité

- **Logs structurés JSON** : chaque service génère des logs au format JSON incluant timestamp, niveau, service_name, trace_id et message.
- **Métriques** : suivi du nombre de requêtes, temps de réponse, taux d'erreur via Laravel Telescope ou un endpoint /metrics.
- **Tracing distribué** : un X-Trace-ID est propagé dans tous les headers HTTP pour retracer le parcours d'une requête à travers les services.

```
// Exemple log structuré JSON
{
  "timestamp": "2025-01-15T10:30:00Z",
  "level": "INFO",
  "service": "task-service",
  "trace_id": "abc-123-xyz",
  "message": "Task created",
  "task_id": 42
}
```

7. Sécurité

La sécurité du système est assurée à plusieurs niveaux :

Niveau	Mécanisme	Détail
Authentification	JWT via Laravel Sanctum	Token Bearer, expiration 24h
Autorisation	Politiques Laravel	Règles par ressource et rôle
Transport	HTTPS/TLS	Chiffrement des communications
Données	Hachage bcrypt	Mots de passe jamais stockés en clair
Validation	Form Requests Laravel	Validation stricte des entrées
Protection	Rate Limiting	60 requêtes/minute par IP
CORS	Headers CORS configurés	Origines autorisées explicites

8. Gestion de la Dette Technique

La dette technique est identifiée, documentée et priorisée pour être traitée progressivement dans les iterations suivantes :

Dette	Impact	Priorité	Plan de résolution
Absence de tests complets	Moyen	Haute	Ajouter PHPUnit + Pest
Logs non centralisés	Faible	Moyenne	Intégrer ELK Stack
Pas de cache Redis	Moyen	Moyenne	Ajouter cache sur les lectures
Documentation Swagger partiel	Faible	Faible	Compléter les annotations
Pas de CI/CD	Moyen	Haute	Configurer GitHub Actions

La gestion de la dette technique suit le principe du Boy Scout Rule : laisser le code dans un meilleur état qu'on ne l'a trouvé. Chaque itération inclut un temps dédié au remboursement de cette dette.

9. Difficultés Rencontrées

Difficultés Techniques

- **Gestion des transactions distribuées** : dans l'architecture microservices, assurer la cohérence des données lors d'opérations impliquant plusieurs services s'est révélé complexe. Le pattern SAGA a été adopté comme solution.
- **Communication inter-services** : la configuration de RabbitMQ et la gestion des messages en cas de panne d'un service ont nécessité une attention particulière concernant les mécanismes de retry et d'acknowledgement.
- **Séparation des bases de données** : supprimer les jointures SQL entre modules pour respecter le principe Database per Service a imposé de repenser certaines requêtes en appels HTTP.

Difficultés Conceptuelles

- **Définition des bounded contexts** : identifier les frontières correctes entre les domaines métier sans créer un couplage excessif a demandé plusieurs itérations de réflexion.
- **Équilibre entre granularité et complexité** : trouver le bon niveau de découpage des microservices sans tomber dans le piège des nano-services trop fragmentés.

10. Conclusion

Ce projet a permis de mettre en pratique l'ensemble des concepts fondamentaux de l'architecture logicielle moderne, depuis la conception d'un monolithe modulaire propre jusqu'à sa transformation en architecture microservices distribuée.

Le Système de Gestion de Tâches développé démontre comment une application peut évoluer progressivement en respectant les principes SOLID, les patterns architecturaux et les bonnes pratiques de qualité logicielle.

Les principales leçons tirées de ce projet sont les suivantes :

- Un monolithe bien structuré est toujours préférable à une architecture microservices mal conçue. La modularité doit précéder la distribution.
- Les ADR sont essentiels pour documenter et justifier les décisions architecturales, facilitant ainsi la maintenance et l'évolution du système.
- La transition vers les microservices ne doit pas être précipitée : elle doit répondre à des besoins réels de scalabilité ou d'indépendance de déploiement.
- L'observabilité (logs, métriques, tracing) est indispensable dans une architecture distribuée et doit être pensée dès la conception.

Ce rapport constitue une base solide pour le développement et l'évolution continue du système, en suivant les principes d'une architecture logicielle maintenable, évolutive et sécurisée.